Words of Concern

Point to be As Allan Bloom has said "Education is the movement from darkness to light". Through this handbook, I have tried to illuminate what might otherwise appear as black boxes to some. In doing so, I have used references from several other authors to synthesize or simplify or elaborate information. This is not possible without omitting details that I deem trivial while dilating the data that I consider relevant to topic. Every effort has been made to avoid errors. In spite of this, some errors might have crept in. Any errors or discrepancies noted maybe brought to my notice which I shall rectify in my next revision.

This handbook is solely for educational purpose and is not for sale. This handbook shall not be reproduced or distributed or used for commercial purposes in any form or by any means.

Thanks,
Deeksha.V.
Interface College of Computer Applications (ICCA)
Davangere



Bibliography

1. Introduction to Design and Analysis of Algorithms, Anany Levitin- 3rd edition, Pearson Reference Books
2. Computer Algorithms/C++, Ellis Horowitz, SatrajSahni and Rajasekaran, 2nd Edition, 2014, Universities Press.
3. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronal L. Rivest, Clifford Stein, 3rd Edition, PHI.
4. Design and Analysis of Algorithms, S. Sridhar, Oxford (Higher Education)

ICCA
Interface College of Computer Applications (ICCA)

Part A

1. Write a Python program that accepts the vertices and edges for a graph and stores it as an adjacency matrix.
2. Write a program to implement a function to print In-Degree, Out-Degree and to display that adjacency matrix.
3. Write a program to implement Tower Hanoi.
4. Write a program to sort a list of N elements using the Selection Sort Technique.
5. Write a program to sort a list of N elements using the Bubble Sort Technique.
6. Write a program to perform the Travelling Salesman Problem.
7. Write a program to implement the Exhaustive Search algorithm for the 0/1 Knapsack.
8. Write a Python program to sort a given set of n integer elements using the Merge Sort algorithm. Execute the program for different input sizes where n > 5000, measure the actual time taken to perform sorting for each input size, and record the results in a table.
9. Write a Python program to sort a given set of n integer elements using the Quick Sort algorithm. Execute the program for different input sizes where n > 5000, measure the actual time taken to perform sorting for each input size, and record the results in a table.
10. Write a program to find the minimum and maximum value in an array using divide and conquer.
11. Write a program to implement the backtracking algorithm for solving problems like N queens.
12. Write a program to implement the greedy algorithm for job sequencing with deadlines.

Part B

1. Write a program to implement the BFS algorithm for a graph
2. Write a program to implement the DFS algorithm for a graph
3. Write a program to sort a list of N elements using the Insertion Sort Technique.
4. Write a program to search a list of N elements using Binary Search Algorithm.
5. Write a program to implement the Linear Search Algorithm.
6. Write a program to find the value of an (where a and n are integers) using both brute-force based algorithm and divide and conquer based algorithm.
7. Write a program to implement the backtracking algorithm for the sum of subsets problem.
8. Write a program to implement the Dynamic Programming algorithm for the 0/1 Knapsack.
9. Write a program to implement the Dynamic Programming algorithm for the Optimal Binary Search Tree Problem.
10. Write a program that implements Prim's algorithm to generate a minimum cost spanning Tree.
11. Write a program that implements Kruskal's algorithm to generate a minimum cost spanning tree.
12. Write a program to find the shortest path to other vertices using Dijkstra’s algorithm

Interface College of Computer Applications (ICCA)

PART A

1. Write a Python program that accepts the vertices and edges for a graph and stores it as an adjacency matrix.

```
node = int(input("Enter the number of nodes"))
adj_matrix = []
for i in range(node):
    row = [0]*node
    adj_matrix.append(row)
for i in range(node):
    data = input(f"Enter the outgoing vertices for vertex {i}:")
    for j in data.split():
        adj_matrix[i][int(j)] = 1
print("\nAdjacency Matrix:")
for row in adj_matrix:
    print(*row)
```

Output

Undirected Graph

```
Enter the number of nodes 5
Enter the outgoing vertices for vertex 0:1 3 4
Enter the outgoing vertices for vertex 1:0 2
Enter the outgoing vertices for vertex 2:1 3
Enter the outgoing vertices for vertex 3:0 2 4
Enter the outgoing vertices for vertex 4:0 3

Adjacency Matrix:
0 1 0 1 1
1 0 1 0 0
0 1 0 1 0
1 0 1 0 1
1 0 0 1 0
```

Interface College of Computer Applications (ICCA)

Directed Graph

```
Enter the number of nodes 5
Enter the outgoing vertices for vertex 0:1 4
Enter the outgoing vertices for vertex 1:2
Enter the outgoing vertices for vertex 2:3
Enter the outgoing vertices for vertex 3:0
Enter the outgoing vertices for vertex 4:3

Adjacency Matrix:
0 1 0 0 1
0 0 1 0 0
0 0 0 1 0
1 0 0 0 0
0 0 0 1 0
```

2. Write a program to implement a function to print In-Degree, Out-Degree and to display adjacency matrix

```
def degree(adj_matrix, node):
    print("\nVertex\tIn-Degree\tOut-Degree")

    for i in range(node):
        in_degree = 0
        out_degree = 0

        for j in range(node):
            out_degree += adj_matrix[i][j]
            in_degree += adj_matrix[j][i]

        print(f"{i}\t{in_degree}\t\t{out_degree}")

def display_adjacency_matrix(adj_matrix):
    print("\nAdjacency Matrix:")
    for row in adj_matrix:
        print(*row)

node = int(input("Enter the number of nodes: "))
adj_matrix = []

for i in range(node):
    row = [0] * node
    adj_matrix.append(row)

for i in range(node):
    data = input(f"Enter the outgoing vertices for vertex {i}:")

    for j in data.split():
        adj_matrix[i][int(j)] = 1
display_adjacency_matrix(adj_matrix)
degree(adj_matrix, node)
```

ICCA
Interface College of Computer Applications (ICCA)

Output

```
Enter the number of nodes: 5
Enter the outgoing vertices for vertex 0:1 4
Enter the outgoing vertices for vertex 1:2
Enter the outgoing vertices for vertex 2:3
Enter the outgoing vertices for vertex 3:0
Enter the outgoing vertices for vertex 4:3

Adjacency Matrix:
0 1 0 0 1
0 0 1 0 0
0 0 0 1 0
1 0 0 0 0
0 0 0 1 0

Vertex  In-Degree       Out-Degree
0       1               2
1       1               1
2       1               1
3       2               1
4       1               1
```

3. Write a program to implement Tower Hanoi.

```
def tower_of_hanoi(n, source, auxiliary, destination):
    if n == 1:
        print("Move disk 1 from", source, "to", destination)
        return

    tower_of_hanoi(n - 1, source, destination, auxiliary)
    print("Move disk", n, "from", source, "to", destination)
    tower_of_hanoi(n - 1, auxiliary, source, destination)

n = int(input("Enter the number of disks: "))
tower_of_hanoi(n, 'A', 'B', 'C')
```

Output

```
Enter the number of disks: 3
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

4. Write a program to sort a list of N elements using the Selection Sort Technique.

```
n = int(input("Enter the number of elements: "))
a = []
print("Enter the elements:")
for i in range(n):
    a.append(int(input()))

for i in range(n - 1):
    min_index = i

    for j in range(i + 1, n):
        if a[j] < a[min_index]:
            min_index = j

    a[i], a[min_index] = a[min_index], a[i]
print("Sorted List:")
print(a)
```

Output

```
Enter the number of elements: 6
Enter the elements:
85
32
14
66
98
40
Sorted List:
[14, 32, 40, 66, 85, 98]
```

ICCA
Interface College of Computer Applications (ICCA)

5. Write a program to sort a list of N elements using the Bubble Sort Technique.

```
n = int(input("Enter the number of elements: "))
a = []
print("Enter the elements:")
for i in range(n):
    a.append(int(input()))

for i in range(n - 1):
    for j in range(n - 1 - i):
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
print("Sorted List:")
print(a)
```

Output

```
Enter the number of elements: 6
Enter the elements:
66
31
45
12
96
74
Sorted List:
[12, 31, 45, 66, 74, 96]
```

6. Write a python program to perform the Travelling Salesman Problem.

```
import itertools  # Used to generate all possible city orders (permutations)
def tsp_brute_force(graph, start=0):
    n = len(graph)          # Number of cities
    other_cities = list(range(1, n))  # All cities except the starting one

    best_tour = None # To store the best tour found
    best_distance = float('inf')  # Start with infinity so any real distance is smaller

    # Try every possible order of the non-start cities
    for perm in itertools.permutations(other_cities):

        # Build a full tour: start → all other cities → back to start
        tour = [start] + list(perm) + [start]

        # Calculate total distance by summing each consecutive city pair
        total = 0
        for i in range(len(tour) - 1):
            total += graph[tour[i]][tour[i+1]]

        if total < best_distance:
            best_tour = tour
            best_distance = total
```

Interface College of Computer Applications (ICCA)

```
    return best_tour, best_distance

# Distance matrix: distances[i][j] = distance between city i and city j
graph = [
    [0,  10, 15, 20],
    [10,  0, 35, 25],
    [15, 35,  0, 30],
    [20, 25, 30,  0]
]

tour, distance = tsp_brute_force(graph)
print("Shortest tour:", tour)
print("Shortest distance:", distance)
```

Output

```
Shortest tour: [0, 1, 3, 2, 0]
Shortest distance: 80
```

7. Write a program to implement the Exhaustive Search algorithm for the 0/1 Knapsack.

```
def knapsack(item, weight, profit):

    global max_profit

    # If all items are considered
    if item == n:

        # Check if the subset is valid
        if weight <= capacity and profit > max_profit:
            max_profit = profit

        return

    # Include the current item
    knapsack(item + 1,weight + weights[item],profit + profits[item])

    # Exclude the current item
    knapsack(item + 1,weight,profit)

# Main Program
n = int(input("Enter the number of items: "))

weights = []
profits = []

for i in range(n):
    print("\nItem", i + 1)
    weight = int(input("Enter Weight: "))
    profit = int(input("Enter Profit: "))
    weights.append(weight)
    profits.append(profit)
```

ICCA Interface College of Computer Applications (ICCA)

```
capacity = int(input("\nEnter Knapsack Capacity: "))
max_profit = 0
knapsack(0, 0, 0)
print("\nMaximum Profit =", max_profit)
```

Output

```
Enter the number of items: 3

Item 1
Enter Weight: 1
Enter Profit: 10

Item 2
Enter Weight: 2
Enter Profit: 15

Item 3
Enter Weight: 3
Enter Profit: 45

Enter Knapsack Capacity: 4

Maximum Profit = 55
```

8. Write a Python program to sort a given set of n integer elements using the Merge Sort algorithm. Execute the program for different input sizes where n > 5000, measure the actual time taken to perform sorting for each input size, and record the results in a table.

```
import random
import time

# Merge Function
def merge(arr, low, mid, high):
    temp = []
    i = low
    j = mid + 1

    while i <= mid and j <= high:
        if arr[i] < arr[j]:
            temp.append(arr[i])
            i = i + 1
        else:
            temp.append(arr[j])
            j = j + 1

    while i <= mid:
        temp.append(arr[i])
        i = i + 1

    while j <= high:
        temp.append(arr[j])
```

ICCA Interface College of Computer Applications (ICCA)

```
        j = j + 1

    k = 0
    for i in range(low, high + 1):
        arr[i] = temp[k]
        k = k + 1

# Merge Sort Function
def mergeSort(arr, low, high):

    if low < high:
        mid = (low + high) // 2

        mergeSort(arr, low, mid)
        mergeSort(arr, mid + 1, high)

        merge(arr, low, mid, high)


# Different input sizes
sizes = [5000, 10000, 15000, 20000]

print("Input Size\tTime Taken (seconds)")

for n in sizes:

    arr = []

    # Generate random numbers
    for i in range(n):
        arr.append(random.randint(1, 100000))
    start = time.time()
    mergeSort(arr, 0, n - 1)
    end = time.time()
    print(n, "\t\t", end - start)
```

ICCA
Interface College of Computer Applications (ICCA)

Output

```
Input Size      Time Taken (seconds)
5000             0.046132326126098 63
10000            0.093301296234130 86
15000            0.133739471435546 88
20000            0.171323299407958 98
100000           1.4751384258270264
```

9. Write a Python program to sort a given set of n integer elements using the Quick Sort algorithm. Execute the program for different input sizes where n > 5000, measure the actual time taken to perform sorting for each input size, and record the results in a table.

```
import random
import time

# Partition function (Pivot = First Element)
def partition(arr, low, high):
    pivot = arr[low]
    i = low + 1
    j = high

    while True:

        while i <= high and arr[i] <= pivot:
            i = i + 1

        while arr[j] > pivot:
            j = j - 1

        if i < j:
            temp = arr[i]
            arr[i] = arr[j]
            arr[j] = temp
        else:
            break

    # Place pivot in its correct position
    temp = arr[low]
    arr[low] = arr[j]
    arr[j] = temp

    return j

# Quick Sort Function
def quickSort(arr, low, high):
    if low < high:
        p = partition(arr, low, high)

        quickSort(arr, low, p - 1)
        quickSort(arr, p + 1, high)


# Different input sizes
sizes = [5000, 10000, 15000, 20000,100000]

print("-----------------------------------------")
print("Input Size (n)\tTime Taken (seconds)")
print("-----------------------------------------")

for n in sizes:

    arr = []
```

Interface College of Computer Applications (ICCA)

```
# Generate random integers
for i in range(n):
    arr.append(random.randint(1, 100000))
start = time.time()
quickSort(arr, 0, n - 1)
end = time.time()
print(n, "\t\t", round(end - start, 6))
```

Output

```
-----------------------------------------
Input Size (n)  Time Taken (seconds)
-----------------------------------------
5000              0.042362
10000             0.056055
15000             0.074122
20000             0.103247
100000            0.53739
```

10. Write a program to find the minimum and maximum value in an array using divide and conquer.

```
def findMinMax(arr, low, high):

    # If there is only one element
    if low == high:
        return arr[low], arr[low]

    # If there are two elements
    if high == low + 1:
        if arr[low] < arr[high]:
            return arr[low], arr[high]
        else:
            return arr[high], arr[low]
    # Divide the array into two halves
    mid = (low + high) // 2

    min1, max1 = findMinMax(arr, low, mid)
    min2, max2 = findMinMax(arr, mid + 1, high)

    # Combine the results
    if min1 < min2:
        minimum = min1
    else:
        minimum = min2

    if max1 > max2:
        maximum = max1
    else:
        maximum = max2
```

ICCA
Interface College of Computer Applications (ICCA)

```
    return minimum, maximum

# Main Program
n = int(input("Enter the number of elements: "))

arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

minimum, maximum = findMinMax(arr, 0, n - 1)

print("Minimum element =", minimum)
print("Maximum element =", maximum)
```

Output

```
Enter the number of elements: 6
Enter the elements:
56
32
85
98
33
44
Minimum element = 32
Maximum element = 98
```

11. Write a program to implement the backtracking algorithm for solving problems like N queens.

```
def printBoard(board, n):
    for i in range(n):
        for j in range(n):
            if board[i][j] == 1:
                print("Q", end=" ")
            else:
                print("_", end=" ")
        print()


# Function to check whether it is safe to place a queen
def isSafe(board, row, col, n):

    # Check left side of the current row
    for i in range(col):
        if board[row][i] == 1:
            return False

    # Check upper left diagonal
    i = row
    j = col
```

ICCA
Interface College of Computer Applications (ICCA)

```
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i = i - 1
        j = j - 1

    # Check lower left diagonal
    i = row
    j = col
    while i < n and j >= 0:
        if board[i][j] == 1:
            return False
        i = i + 1
        j = j - 1

    return True


# Backtracking function
def solveNQueens(board, col, n):

    # All queens are placed
    if col == n:
        return True

    # Try placing queen in every row
    for i in range(n):

        if isSafe(board, i, col, n):

            board[i][col] = 1

            if solveNQueens(board, col + 1, n):
                return True

            # Backtrack
            board[i][col] = 0

    return False

# Main Program
n = int(input("Enter the value of N: "))
board = []
for i in range(n):
    row = []
    for j in range(n):
        row.append(0)
    board.append(row)

if solveNQueens(board, 0, n):
    print("\nSolution:")
    printBoard(board, n)
else:
    print("Solution does not exist.")
```

ICCA
Interface College of Computer Applications (ICCA)

Output

```
Enter the value of N: 6

Solution:
_ _ _ Q _ _
Q _ _ _ _ _
_ _ _ _ Q _
_ Q _ _ _ _
_ _ _ _ _ Q
_ _ Q _ _ _
```

12. Write a program to implement the greedy algorithm for job sequencing with deadlines.

```
n = int(input("Enter number of jobs: "))

jobs = []

for i in range(n):
    print("\nJob", i + 1)

    deadline = int(input("Enter Deadline: "))
    profit = int(input("Enter Profit: "))

    jobs.append(["Job" + str(i + 1), deadline, profit])

# Sort jobs according to profit
jobs.sort(key=lambda x: x[2], reverse=True)

# Find maximum deadline
max_deadline = jobs[0][1]

for i in range(1, n):
    if jobs[i][1] > max_deadline:
        max_deadline = jobs[i][1]

# Create slots
slots = []

for i in range(max_deadline):
    slots.append(None)

total_profit = 0

# Schedule jobs
for job in jobs:
    for i in range(job[1] - 1, -1, -1):
        if slots[i] is None:
            slots[i] = job[0]
            total_profit += job[2]
            break

print("\nSelected Jobs:")
```

ICCA Interface College of Computer Applications (ICCA)

```
for job in slots:
    if job is not None:
        print(job)
print("Maximum Profit =", total_profit)
```

Output

```
Enter number of jobs: 5

Job 1
Enter Deadline: 2
Enter Profit: 100

Job 2
Enter Deadline: 2
Enter Profit: 50

Job 3
Enter Deadline: 1
Enter Profit: 200

Job 4
Enter Deadline: 1
Enter Profit: 250

Job 5
Enter Deadline: 3
Enter Profit: 60

Selected Jobs:
Job4
Job1
Job5
Maximum Profit = 410
```

Interface College of Computer Applications (ICCA)

PART B

1. Write a program to implement the BFS algorithm for a graph

```
def bfs(graph, start):
    visited = set()
    queue = [start]

    while queue:
        vertex = queue.pop(0)
        if vertex not in visited:
            visited.add(vertex)
            print(vertex, end=" ")

            for neighbor in graph[vertex]:
                if neighbor not in visited:
                    queue.append(neighbor)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}
start_node = 'A'
print("Breadth-First Search:")
bfs(graph, start_node)
```

Output

```
Breadth-First Search:
A B C D E F
```

2. Write a program to implement the DFS algorithm for a graph

```
def dfs(graph, start):
    visited = set()
    stack = [start]

    while stack:
        vertex = stack.pop()
        if vertex not in visited:
            visited.add(vertex)
            print(vertex, end=" ")

            for neighbor in graph[vertex]:
                if neighbor not in visited:
                    stack.append(neighbor)

    return visited
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
```

Interface College of Computer Applications (ICCA)

```
    'E': ['F'],
    'F': []
}
start_vertex = 'A'
print("Depth-First Search traversal:")
dfs(graph, start_vertex)
```

Output

```
Depth-First Search traversal:
A C F B E D
```

3. Write a program to sort a list of N elements using the Insertion Sort Technique.

```
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

for i in range(1, n):

    key = arr[i]
    j = i - 1

    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j = j - 1

    arr[j + 1] = key

print("\nSorted List:")
for i in range(n):
    print(arr[i], end=" ")
```

Output

```
Enter the number of elements: 5
Enter the elements:
15
20
11
96
33

Sorted List:
11 15 20 33 96
```

4. Write a program to search a list of N elements using Binary Search Algorithm.

```
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements in sorted order:")
for i in range(n):
    arr.append(int(input()))
```

ICCA Computer Applications (ICCA)

```
key = int(input("Enter the element to search: "))

low = 0
high = n - 1
found = False

while low <= high:

    mid = (low + high) // 2

    if arr[mid] == key:
        print("Element found at index", mid)
        found = True
        break

    elif key < arr[mid]:
        high = mid - 1

    else:
        low = mid + 1

if found == False:
    print("Element not found")
```

Output

```
Enter the number of elements: 6
Enter the elements in sorted order:
15
23
33
46
59
68
Enter the element to search: 46
Element found at index 3
```

5. Write a program to implement the Linear Search Algorithm.

```
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

key = int(input("Enter the element to search: "))

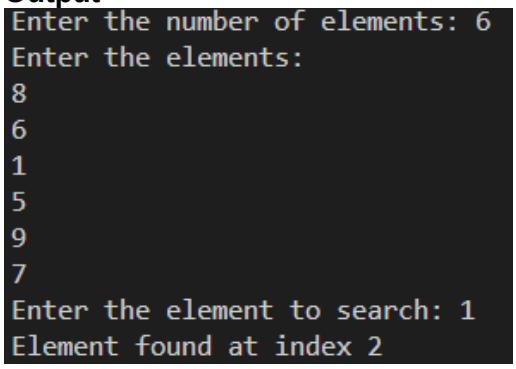
found = False

for i in range(n):
    if arr[i] == key:
        print("Element found at index", i)
        found = True
```

```
        break

if found == False:
    print("Element not found")
```

Output



6. Write a program to find the value of a^n (where a and n are integers) using both brute-force based algorithm and divide and conquer based algorithm.

```
def bruteForce(a, n):

    result = 1

    for i in range(n):
        result = result * a

    return result

def divideConquer(a, n):

    if n == 0:
        return 1

    if n == 1:
        return a
    half = divideConquer(a, n // 2)

    if n % 2 == 0:
        return half * half
    else:
        return a * half * half

a = int(input("Enter the value of a: "))
n = int(input("Enter the value of n: "))

print("\nBrute Force Result =", bruteForce(a, n))
print("Divide and Conquer Result =", divideConquer(a, n))
```

ICCA
Interface College of Computer Applications (ICCA)

Output

```
Enter the value of a: 2
Enter the value of n: 8

Brute Force Result = 256
Divide and Conquer Result = 256
```

7. Write a program to implement the backtracking algorithm for the sum of subsets problem.

```
def subset(index, current_sum):

    if current_sum == target:
        print("Subset Found:", selected)
        return

    if index == n or current_sum > target:
        return

    # Include the current element
    selected.append(arr[index])
    subset(index + 1, current_sum + arr[index])

    # Backtrack
    selected.pop()

    # Exclude the current element
    subset(index + 1, current_sum)

n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
target = int(input("Enter the target sum: "))
selected = []
subset(0, 0)
```

ICCA
Interface College of Computer Applications (ICCA)

Output

```
Enter the number of elements: 5
Enter the elements:
5
10
20
13
15
Enter the target sum: 25
Subset Found: [5, 20]
Subset Found: [10, 15]
```

8. Write a program to implement the Dynamic Programming algorithm for the 0/1 Knapsack.

```
n = int(input("Enter the number of items: "))

weights = []
profits = []

for i in range(n):
    print("\nItem", i + 1)
    weights.append(int(input("Enter Weight: ")))
    profits.append(int(input("Enter Profit: ")))

capacity = int(input("\nEnter Knapsack Capacity: "))

# Create DP array
dp = []

for i in range(capacity + 1):
    dp.append(0)

# Build DP array
for i in range(n):

    for w in range(capacity, weights[i] - 1, -1):

        if dp[w] < dp[w - weights[i]] + profits[i]:
            dp[w] = dp[w - weights[i]] + profits[i]

print("\nMaximum Profit =", dp[capacity])
```

Output

```
Enter the number of items: 3

Item 1
Enter Weight: 8
Enter Profit: 100

Item 2
Enter Weight: 2
Enter Profit: 50

Item 3
Enter Weight: 5
Enter Profit: 200

Enter Knapsack Capacity: 10

Maximum Profit = 250
```

ICCA
puter Applications (ICCA)

9. Write a program to implement the Dynamic Programming algorithm for the Optimal Binary Search Tree Problem.

```
def optimalBST(keys, freq, n):
    # Create DP tables
    cost = []
    freq_sum = []

    for i in range(n + 1):
        cost_row = []
        freq_row = []

        for j in range(n + 1):
            cost_row.append(0)
            freq_row.append(0)

        cost.append(cost_row)
        freq_sum.append(freq_row)

    # Initialize frequency sum and cost tables
    for i in range(n):
        freq_sum[i][i + 1] = freq[i]
        cost[i][i + 1] = freq[i]

    # Fill DP tables
    for length in range(2, n + 1):

        for i in range(n - length + 1):

            j = i + length

            freq_sum[i][j] = freq_sum[i][j - 1] + freq[j - 1]

            cost[i][j] = 999999

            # Find minimum cost
            for r in range(i, j):
                current_cost = cost[i][r] + cost[r + 1][j] + freq_sum[i][j]

                if current_cost < cost[i][j]:
                    cost[i][j] = current_cost

    print("Optimal Cost =", cost[0][n])

n = int(input("Enter the number of keys: "))

keys = []
freq = []

print("Enter the keys:")
for i in range(n):
    keys.append(int(input()))

print("Enter the frequencies:")
```

ICCA
Interface College of Computer Applications (ICCA)

```
for i in range(n):
    freq.append(int(input()))

optimalBST(keys, freq, n)
```

Output

```
Enter the number of keys: 3
Enter the keys:
10
20
30
Enter the frequencies:
34
8
50
Optimal Cost = 142
```

10. Write a program that implements Prim's algorithm to generate a minimum cost spanning Tree.

```
graph = [
    [0,3,0,0,6,5],
    [3,0,1,0,0,4],
    [0,1,0,6,0,4],
    [0,0,6,0,8,5],
    [6,0,0,8,0,2],
    [5,4,4,5,2,0]
]

n = 6

visited = [False] * n
visited[0] = True

edge_count = 0
minimum_cost = 0

print("Edges in the Minimum Spanning Tree:")

while edge_count < n - 1:

    minimum = 999999
    x = 0
    y = 0

    for i in range(n):
        if visited[i]:

            for j in range(n):
                if not visited[j] and graph[i][j] != 0:

                    if graph[i][j] < minimum:
```

ICCA Interface College of Computer Applications (ICCA)

```
                minimum = graph[i][j]
                x = i
                y = j

    print(x, "--", y, "=", graph[x][y])

    minimum_cost += graph[x][y]
    visited[y] = True
    edge_count += 1

print("\nMinimum Cost =", minimum_cost)
```

Output

```
Edges in the Minimum Spanning Tree:
0 -- 1 = 3
1 -- 2 = 1
1 -- 5 = 4
5 -- 4 = 2
5 -- 3 = 5

Minimum Cost = 15
```

11. Write a program that implements Kruskal's algorithm to generate a minimum cost spanning tree.

```
graph = [
    [0, 3, 0, 0, 6, 5],
    [3, 0, 1, 0, 0, 4],
    [0, 1, 0, 6, 0, 4],
    [0, 0, 6, 0, 8, 5],
    [6, 0, 0, 8, 0, 2],
    [5, 4, 4, 5, 2, 0]
]

n = len(graph)

# Convert adjacency matrix to edge list
edges = []

for i in range(n):
    for j in range(i + 1, n):
        if graph[i][j] != 0:
            edges.append((i, j, graph[i][j]))

# Sort edges according to weight
edges.sort(key=lambda edge: edge[2])

parent = []

for i in range(n):
    parent.append(i)

def find(vertex):
```

ICCA Interface College of Computer Applications (ICCA)

```
    while parent[vertex] != vertex:
        vertex = parent[vertex]
    return vertex

def union(vertex1, vertex2):
    root1 = find(vertex1)
    root2 = find(vertex2)
    parent[root1] = root2

minimum_cost = 0
edge_count = 0

print("Edges in the Minimum Spanning Tree:")

for edge in edges:

    u = edge[0]
    v = edge[1]
    weight = edge[2]

    if find(u) != find(v):

        union(u, v)

        print(u, "--", v, "=", weight)

        minimum_cost = minimum_cost + weight
        edge_count = edge_count + 1

        if edge_count == n - 1:
            break
print("\nMinimum Cost =", minimum_cost)
```

Output

```
Edges in the Minimum Spanning Tree:
1 -- 2 = 1
4 -- 5 = 2
0 -- 1 = 3
1 -- 5 = 4
3 -- 5 = 5

Minimum Cost = 15
```

12. Write a program to find the shortest path to other vertices using Dijkstra's algorithm.

```
graph = [
    [0, 3, 0, 7, 0],
    [3, 0, 4, 2, 0],
    [0, 4, 0, 5, 6],
    [7, 2, 5, 0, 4],
    [0, 0, 6, 4, 0]
]
```

```
n = len(graph)

distance = []
visited = []

for i in range(n):
    distance.append(float('inf'))
    visited.append(False)

source = 0
distance[source] = 0

for i in range(n):
    minimum = float('inf')
    u = -1
    # Find the unvisited vertex with minimum distance
    for j in range(n):
        if not visited[j] and distance[j] < minimum:
            minimum = distance[j]
            u = j
    visited[u] = True

    # Update distances of adjacent vertices
    for v in range(n):
        if graph[u][v] != 0 and not visited[v]:

            new_distance = distance[u] + graph[u][v]

            if new_distance < distance[v]:
                distance[v] = new_distance

for i in range(1,n):
    print("Shortest distances from source 0 to vertex", i, ":", distance[i])
```

Output

```
Shortest distances from source 0 to vertex 1 : 3
Shortest distances from source 0 to vertex 2 : 7
Shortest distances from source 0 to vertex 3 : 5
Shortest distances from source 0 to vertex 4 : 9
```

ICCA
ations (ICCA)